Instructions
Week 4: Tidyverse Continued

GOVT16/QSS17: Data Visualization (Prof. Cooper)

Week 4

- Date: April 21st - April 28th (Due Tuesday, April 28th, 11:59 PM)

Preparation

1. During this term, you will use a specific folder within your computer – your "working directory" – where you will save your R scripts, data, figures, and other documents. To make this folder, open RStudio and go to "File" > "New Project". Then, choose "New Directory", and make a new "project" (i.e., a new directory) somewhere on your computer. The project name, which is the name of your working directory, can be anything. Check that the new folder is generated with a new `.Rproj` file. As long as all of your files for this course are saved in this working directory, you need to make this `.Rproj` file once. When you work on your own project after this course, however, you should specify a different working directory with a new `.Rproj` file for each project.

What should you do in Week 4?

- In RStudio, choose "File" > "New File" > "R Script". Then, save the file as "Week04_script.R" *within the "Week04" sub-folder.*
- Work on Exercises below.
- You should upload your R script by 11:59 pm, Tuesday, April 28th to Canvas ("Assignments" > "In-Class Exercises" > "Week 4 Assignment").

Notes:

- Normally I would ask you to not write or type your answers *within your script.* But feel free to write some comments (with #) for your own memo. At this stage I encourage you to write reasonably extensive comments in your own version of your scripts so that you remember what it is you did and why you did it.

Scripts to be read by others, under normal circumstances, should not be quite so heavyhanded with comments.

- Each time I ask you to create a new object, please follow the creation of that object by printing the object. Under normal circumstances you would not print out every object in your R script, whether using proper workflow on your own or sharing scripts with other researchers and collaborators. It will be useful for all of us *in the context of this course*. You can check your work constantly by seeing the output and I can grade more efficiently by seeing the output.

- If I use a particular verb or reference a particular operator or function that you do not immediately recognize, there are many resources out there to help you to discover the solution. The process of search and discovery for basic coding solutions is a part of programming, and **R** has many online resources for you. Google is your friend, along with a very large number of sources for **R** support.

Exercises

Set up the top of your R script with your name, the name of the course, the term, and the date using comments. There are different style guides for comments. For your homeworks, please use `#` (followed by one space) to add a comment to your code. Begin with `##` (followed by one space) for entirely commented lines.

```
> ## Data Visualization (GOVT16-QSS17) Spring 2026
> ## Tidyverse Continued
> ##
> ## Name: YOUR NAME
> ## Date: April 21 - April 28
```

Read the `movie` and `wiid` datasets into **RStudio** using the `read_csv` command you should have learned about this week.
For some of your work today you might have to deal with missing data points. That's life as a researcher. There's an argument to help, however. For now we are going to remove the "NA" missing data points inside the statistical functions like `mean`, `median`, `min`, or `max` with the `na.rm = TRUE` argument. For plots, FIX ALL LABELS. For all the problems, USE PIPING. USE ALL HEX CODE COLORS UNLESS SPECIFIED OTHERWISE.

1. **Some Counting, Summarizing, and Filtering Options with your IMDB Movies Data**

   (a) Let's start with something simple. Practice with the `skim` function. Skim the `movie` dataset with `skim()`. You will need to load a separate tidyverse package called `skimr`; install it and then call the library. This function provides summary statistics for *every* variable in the data set.

   (b) Show me all the different languages represented in the `movie` dataset with the `distinct` function.

   (c) Count the the movies (using `count`) by `language`. Sort them, please.

   (d) Count the movies both by `language and color`. Sort them, please.

   (e) I am going to introduce a function to you for this problem, the logical operator `%in%`. It determines whether the elements of the first vector are also contained in the second vector. To practice with it, make a vector called `vector1` that gets values 1 through 10 in sequence. Then create a `vector2` that gets the values 1, 4, and 11.

   (f) To practice and understand the `%in%`, type `vector1 %in% vector2` and see what R prints. Then, do it in reverse, typing `vector2 %in% vector1` and printing it. Pay attention to the difference; this is how you understand functions.

(g) Obviously, most of the movies in this database are in English. Boring! Using a plot/picture and not words, count the number of films in French, Spanish, Hindi, and Mandarin. To do so, use the `%in%` operator to filter for those lanaguages only and then provide a bar plot.

2. Pulling Apart and Creating String/Character Variables

   (a) In the next few problems I am inviting you to use the `stringr` package inside `tidyverse`. This package helps you work with character or string variables to find patterns that you might want to highlight, analyze, subset, or change altogether. There is an impressive set of functions inside `stringr` for these purposes, pretty much all set up in a `str_SOMETHING()` format, where `SOMETHING` is replaced with a number of verbs and adjectives. These functions work to apply rules and find patterns *inside* vectors. Download the **RStudio** cheatsheet on `stringr` if you have not yet done so.

   (b) First, let's go back to our `count` function. I want to know if different genres have different average IMDB scores. To start, Let's just count the different genres in the dataset with the `count` function.

   (c) Wow, that's messy. It makes sense that a movie might be in multiple categories, but the coding done here is kind of ridiculous. Let's find which movies are considered action movies. Use `str_detect()` somehow to create a new variable `action` and then count the action versus non-action movies. There's a really useful function called `ifelse` you might consider, although it is not necessary. You could also try `case_when`. Once you have the new variable, perform a count.

   (d) Perform another `count` like above, but do so for animated movies.

   (e) I want to know whether comedies are rated higher than dramas on average in their IMDB scores over time. More generally, I want to know how the IMDB scores for the average comedy or drama evolve by year. Keep in mind that some movies will be considered both, and that is no problem. I suggest you simply have a "Comedy-Drama" category for those. Please show me with a fairly transparent scatterplot underneath layer and with a smoother line on top by category. Use your own hex code colors. Move the legend to the bottom of the panel. Fix all labels.

3. The IMDB database we have been using is a fairly small sample of what is currently available. For this final series of problems you will be using a new, updated set of data files I have provided you.

   (a) The IMDB database has *millions* of observations and is more representative of the broader array of options available to consumers. I have provided two rather large datasets for you that have been freshly pulled from IMDB. Read in the `imdb.tsv` and `ratings.tsv` files from your Canvas Files/Assignments/Week 4 Module. The `.tsv` files read just like `.csv`

files; the `readr` command for importing these files is `read_tsv()`. Just for your information, `tsv` stands for "tab-separated values," while `csv` stands for "comma-separated values."

(b) Let's start with the `imdb.tsv` file. I wonder if the genres variable has improved since a few years ago. Let's investigate. Use a simple `count` function to do so.

(c) We are going to need to `join` the two datasets together to be able to get what we want for analysis and visualization. A suite of these functions exists in `dplyr`. I would start with `full_join`. Use the helper file to investigate; all you need is one ID variable from both data sets on which to match. Join the `ratings.tsv` and `imdb.tsv` datasets using the film ID in the `by` argument.

(d) There seem to be many observations that have no average IMDB scores (now called `AverageScore`). I think we should look to see why. Use the `is.na` function somehow to figure this out. Specifically, let's try investigating the variable `titleType`. Show me how many missing values exist within each title type.

(e) Ahh! Most, though certainly not all, of the missing IMDB scores are from TV episodes. For the final plot you will subset to films with IMDB scores. Let's create a line plot comparing scores of two of the film title types, full-length movies and "shorts," films shorter than 40 minutes including credits. Instead of working with means, however, let's make the underneath layer is a bit different. Please estimate the following quantities: the median IMDB score (don't be confused that the IMDB score is also an "average" score across individuals who vote), the 25th percentile (also known as the `quantile` at .25), and the 75th percentile (also known as the `quantile` at .75). Create these in one mutate function (which can, of course, contain many new variables).

   i. Use a geom called a `geom_segment` for your underneath layer and a smoother line as your top layer for the plot. Remember, we are comparing full-length films against shorts. The underneath segments should be vertical bars for each year, ranging from the 25th to the 75th percentile, representing the IQR (interquartile range) for each year. Set the alpha parameter to about 0.4 for the segments.

   ii. The top layer of the plot should be smoother lines. Please shrink down the `span` of the smoother lines to about 0.3.

   iii. Set one color, for short films, to a relatively dark, mustard-like yellow using hex codes. Pick a good medium-intensity blue for full-length films.

   iv. Move the legend inside the plot to negative space on the bottom right of the plotting panel. Fix all labels.